

Le serveur tourne. Maintenant, **on lance.**

Analyse de ce qui a été mis en place (builds 1581 à 1600).
4 urgences avant le lancement, puis 20 optimisations
et nouveautés. Pour chaque idée : le constat, l'idée,
puis le ou les prompts à coller dans Claude Code.

Où on en est (build 1604)

Gros pas en avant : la branche viralité est dans main, et le **serveur léger Cloudflare** remplace les Cloud Functions à 0 €. Le parrainage, le push, les défis, le webhook PayPal, la suppression Cloudinary et les relances de fin d'accès **tourment**. Le plan Blaze n'est plus nécessaire.

La revue du serveur a trouvé **deux bugs graves à corriger avant d'ouvrir les vannes** : la suppression des médias ne peut pas marcher depuis le navigateur (la requête préalable plante), et n'importe quel compte peut supprimer les photos d'un autre. Plusieurs cas PayPal font aussi **perdre de l'argent ou des récompenses** en silence. Et la page d'accueil annonce **d'autres prix** que l'app. Tout est dans la partie 0.

Les 22 idées de la V3

Idée V3	État	Ce qui reste
01 Fusion et builds	Fait	—
02 Serveur, parrainage	Fait (Cloudflare)	Aperçus de liens personnalisés non portés sur le serveur léger.
03 Notifications justes	Pas fait	Série à vie aux inactifs, bilan sans coach, carte 2 boutons, pas de plafond commun.
04 Accrocs du partage	Pas fait	iPhone sans copie de lien, nom de fichier, faute « RepCore Avec », bouton record.
05 « Thomas t'invite »	Pas fait	Le prénom est stocké mais jamais affiché à l'arrivée.
06 Format post 4:5	Partiel	Coach et avant/après seulement ; pas de carrousel Wrapped.
07 Légende + #RepCore	Pas fait	—
08 Export vidéo	Pas fait	—
09 Parrainage visible	Partiel	Écran et rappel oui ; pas de carte visuelle, pas sur rang/série/Wrapped.
10 Page publique riche	Partiel	Emblème oui ; badges en texte, records sans valeur, pas d'invitation à l'activer.
11 Carte d'athlète /99	Pas fait	—
12 Une fête par séance	Pas fait	—
13 7 premiers jours	Pas fait	—
14 Record à portée	Pas fait	—
15 Check-in du matin	Pas fait	—
16 Relance des inactifs	Pas fait	—
17 Duels entre potes	Pas fait	—
18 Sous-niveaux, XP serveur	Pas fait	Maintenant faisable avec le serveur léger.
19 Éditions limitées	Pas fait	—
20 Kit coach / ambassadeur	Pas fait	Les codes ambassadeurs existent.
21 Calendrier de contenu	À faire hors code	Prompt ChatGPT en fin de document.
22 Cohortes de rétention	Pas fait	L'entonnoir par source existe.

Ordre conseillé : 01 → 02 → 03 → 07 (bloquants et argent) → 04 → 05 → 06 → 08, puis 09 → 13 → 10 → 11 pour le lancement Instagram. Les idées déjà décrites en V3 et toujours à faire sont reprises en version courte, adaptées au serveur léger (cloudflare/src) à la place de functions/index.js.

0. À corriger avant le lancement

IDÉE 01 · BLOQUANT · EFFORT MINIME

La requête préalable du navigateur plante le serveur

CONSTAT

Dans `cloudflare/src/index.js`, la réponse à la requête `OPTIONS` est `new Response("", {status:204})`. Une réponse 204 ne peut pas avoir de corps, même vide : le constructeur lève une erreur (vérifié), hors du `try`. Le navigateur envoie cette requête avant chaque appel `/fn` qui porte un jeton : **cloudinaryDestroy échoue donc toujours**, et les suppressions restent en file pour toujours. Les tests ne le voient pas car ils appellent la fonction directement.

L'IDÉE

Une ligne : `null` au lieu de `"`. Plus un test qui passe par `fetch` avec `OPTIONS`, pour que ça ne revienne pas.

PROMPT CLAUDE CODE

Dans `cloudflare/src/index.js` :

1. La réponse aux requêtes `OPTIONS` doit être `new Response(null, {status:204, headers:CORS})` : une réponse 204 ne peut pas avoir de corps, même '' (le constructeur lève « Invalid response status code 204 »). Adapte le helper `reponse()` pour qu'il passe `null` quand le statut est 204 ou 304.
2. Mets toute la fonction `fetch` dans le `try` (`OPTIONS` compris) pour qu'aucune exception ne sorte sans en-têtes `CORS`.
3. Ajoute dans `cloudflare/test` un test qui appelle le handler `fetch` exporté avec une `Request OPTIONS` sur `/fn/cloudinaryDestroy` et vérifie 204 + en-têtes `CORS`, puis un `POST` sans jeton qui doit rendre 401 avec `CORS`.
4. Ajoute un script de test unique (`npm test` ou `node --test` sur les fichiers `.test.mjs` un par un) et documente-le dans `cloudflare/README.md`.
5. Après déploiement, vérifie depuis l'app qu'une suppression de photo de bilan vide bien la file (`_cldIndispo` ne doit plus passer à `true`).

IDÉE 02 · SÉCURITÉ · EFFORT MOYEN

N'importe qui peut supprimer les photos d'un autre

CONSTAT

`cloudflare/src/medias.js` vérifie la propriété en comparant le dossier de l'URL avec `users/<moi>/id`. Or chaque utilisateur peut écrire librement dans son propre dossier, y compris ce champ `id` (la règle ne fait que vérifier que c'est une chaîne). Un malin recopie l'id d'une victime (visible dans l'URL de ses médias) et supprime ses photos de bilan. Même chose pour la branche « coach », car `role` et `id` sont écrits par le client.

L'IDÉE

Fonder la propriété sur ce que le client ne peut pas écrire : l'identité du jeton `Firestore`, et un `index id` → compte écrit une seule fois. Et ne plus accepter le `cloudName` envoyé par le client.

PROMPT CLAUDE CODE

Sécurise la suppression `Cloudinary` (`cloudflare/src/medias.js`, `database.rules.json`, `app/rc-core*.js`) :

1. Crée un `index /medias_proprio/<id>` -> `<emailKey>`, écrit une seule fois : règle `".write": "auth != null && !data.exists() && newData.val() === auth.token.email.replace('.', ',')"`. Le client l'écrit à la création du compte (et une fois au démarrage pour les comptes existants, si absent).
2. Dans `users/$emailKey`, rends `id` non modifiable une fois posé : `".validate": "newData.isString() && newData.val().length < 40 && (!data.exists() || data.val() === newData.val())"`.
3. `medias.js` : le propriétaire d'un `publicId` `repcore/<id>/...` est lu dans `/medias_proprio/<id>` ; l'appelant doit être ce compte (email du jeton vérifié), ou son coach, vérifié par `users/<proprio>/coachEmailKey === compte de l'appelant` (ce champ est écrit par le coach au rattachement : vérifie que l'athlète ne peut pas le remplacer par n'importe qui ; sinon utilise la liste `coachs/<coach>/clients`).
4. N'accepte plus `cloudName` venant du client : prends-le dans les secrets du worker.
5. Tests dans `cloudflare/test` : `id usurpé` -> 403, `coach usurpé` -> 403, `propriétaire` -> OK, `coach réel` -> OK.
6. Les réponses 400/403 définitives doivent sortir la suppression de la file côté client (aujourd'hui elles sont rejouées à chaque démarrage).

PayPal : ne plus perdre de paiements ni de récompenses

CONSTAT

Le webhook marque l'événement « traité » **avant** de le traiter : si quoi que ce soit échoue, PayPal renvoie, on répond « déjà traité », et l'annulation ou le paiement n'est jamais appliqué. Les premiers paiements qui arrivent avant que l'abonnement soit relié au compte sont jetés (mois du parrain, commission, attribution perdus). Plusieurs cas d'ordre : un paiement tardif après annulation rouvre l'accès à vie ; une suspension puis une annulation effacent les mois offerts ; résilier un ancien abonnement coupe le nouveau ; un coach qui repaie reste en « libre ». Les achats de programme (capture) ne sont reliés à personne.

L'IDÉE

Un webhook qui ne marque qu'après succès, qui garde les orphelins pour les rejouer, et qui demande l'état réel de l'abonnement à PayPal avant d'ouvrir ou de fermer quoi que ce soit.

PROMPT CLAUDE CODE

```

Fiabilise le webhook PayPal (cloudflare/src/paypal.js, métier.js, app/rc-core.*.js) :
1. Idempotence : écris paypal_evenements/<id> = {etat:'en_cours', at} avant, et {etat:'fait'} seulement après succès. Un 'en_cours' de plus de 10 min est repris par le prochain envoi de PayPal. En cas d'erreur, réponds 500 (PayPal renverra).
2. Orphelins : si l'abonnement n'est pas encore relié à un compte, range l'événement dans paypal_orphelins/<abo> et rejoue-le dans indexer() dès que le lien est fait. Ne réponds jamais « sans_compte » en le perdant.
3. Liaison sûre : crée les abonnements avec custom_id = <emailKey> côté client (subscription.create), et dans indexer() vérifie sub.custom_id === compte avant de relier. Même chose pour les achats de programme : mets le compte dans custom_id (ou purchase_units) et relis la commande par l'API pour PAYMENT.CAPTURE.COMPLETED.
4. Avant d'ouvrir l'accès sur un paiement, GET de l'abonnement : n'ouvre que si status === 'ACTIVE' et si c'est l'abonnement courant du dossier (paypalSubscriptionId). Avant de fermer, ignore un abonnement qui n'est pas le courant.
5. fermerALaFin : fin = max(fin existante, next_billing_time) et garde moisEnReserve jusqu'à la fin effective (ne remets pas à 0 dans une mise à jour séparée : transaction).
6. paiement() d'un coach : restaure coachPlan et coachSubActive selon le plan payé.
7. crediterMoisOffert, branche « fin reculée » : seulement si abonnement.finAccesPaypal existe vraiment.
8. Vérifie montant, devise et plan du paiement contre la table OFFRES avant de le compter comme « premier paiement ».
9. Mets le jeton OAuth PayPal en cache (KV ou mémoire du worker, jusqu'à expiration) : 2 sous-requêtes de moins par webhook.
10. Tests dans cloudflare/test/paypal.test.mjs pour chaque cas : double envoi, erreur puis renvoi, orphelin rejoué, paiement après annulation, suspension + annulation avec mois en réserve, ancien abonnement annulé, coach qui repaie.

```

Remboursements et litiges

CONSTAT

Un remboursement ne fait que marquer la commission de l'ambassadeur. Le mois offert au parrain reste acquis, l'accès du remboursé continue, et les rétrofacturations et litiges (REVERSED, CUSTOMER.DISPUTE) sont ignorés : les commissions restent dues. C'est la porte ouverte à « je m'abonne, je me fais rembourser, mon pote garde son mois ».

L'IDÉE

Un remboursement ou un litige annule ce que le paiement avait déclenché, et ferme l'accès à la fin de la période remboursée.

PROMPT CLAUDE CODE

Dans cloudflare/src/metier.js et paypal.js :

1. Gère PAYMENT.SALE.REFUNDED, PAYMENT.CAPTURE.REFUNDED, PAYMENT.SALE.REVERSED et CUSTOMER.DISPUTE.CREATED/RESOLVED.
2. Sur remboursement total ou rétrofacturation d'un premier paiement : retire le mois offert au parrain s'il n'est pas encore consommé (sinon note une dette consommée sur son prochain mois), passe la commission ambassadeur en « annulée », retire l'état « payant » de l'attribution, et ferme l'accès du remboursé à la date du remboursement.
3. Litige ouvert : commission « suspendue » ; litige perdu = comme un remboursement ; gagné = rien.
4. Remboursement partiel : ne touche qu'à la commission, au prorata.
5. Journal lisible dans l'écran admin (qui, quoi, pourquoi) et un push à Kevin pour chaque litige.
6. Tests pour chaque cas.

IDÉE 05 · ARGENT · EFFORT MOYEN

Les droits écrits par le serveur, pas par le client

CONSTAT

Aujourd'hui l'état payé ou expiré (accessExpiry, paymentStatus, abonnement) est écrit dans users/<compte>, que l'utilisateur peut modifier lui-même : un abonné résilié peut effacer sa date de fin depuis la console. Le parrainage se fie aussi à des champs écrits par le client (createdAt, droits.source vide pour les abonnés PayPal) : un ancien abonné peut « rajeunir » son compte et faire gagner un mois à un ami.

L'IDÉE

Le serveur léger a maintenant un accès administrateur : c'est lui qui doit écrire les droits dans droits/ (le seul nœud que le client ne peut pas modifier), et l'app lit ce nœud en priorité.

PROMPT CLAUDE CODE

Fais porter les droits par le serveur léger :

1. Dans cloudflare/src/paypal.js, à chaque changement (paiement, annulation, fin, remboursement), écris droits/<compte> = {palier, echeance, source:'paypal', abo} en plus de l'état actuel dans le dossier.
2. database.rules.json : droits/ reste en lecture pour le titulaire, son coach et le créateur, en écriture pour le créateur seul (le worker écrit avec son accès administrateur). Vérifie que rien d'autre ne l'ouvre.
3. Client : les droits effectifs se lisent d'abord dans droits/ (source paypal), puis dans l'ancien modèle ; supprime la possibilité de s'accorder un accès en modifiant son propre dossier (garde _palierHerite pour la transition).
4. Parrainage : dejaPaye se base sur paypal_premiers, paypalSubscriptionId ou droits.source, pas sur createdAt. Script ponctuel pour remplir paypal_premiers avec les abonnés déjà payants.
5. Remplace le secret de base de données historique (FIREBASE_DB_SECRET, accès total, envoyé dans l'URL) par un jeton OAuth de compte de service, dans l'en-tête Authorization. Documente la rotation dans cloudflare/README.md.
6. Tests : un client qui efface accessExpiry garde un accès fermé ; un ancien payeur ne déclenche pas de mois parrain.

La file d'événements et les quotas du plan gratuit

CONSTAT

Le plan gratuit de Cloudflare autorise 50 sous-requêtes par exécution. Un défi publié chez un coach de 15 athlètes les dépasse : l'événement échoue, reste en tête de file, et **bloque toute la file** (parrainages et abonnements compris), chaque minute, pour toujours. Même risque pour les tâches quotidiennes. N'importe qui peut remplir /evenements sans limite, et /reveil, /arrivee, /amb-clic et /sante?cles=1 sont ouverts à tous (/reveil en boucle fait traiter deux fois les mêmes événements).

L'IDÉE

Des tâches découpées en petits morceaux, une file qui ne se bloque jamais sur un élément, des écritures limitées par les règles, et des points d'entrée protégés.

PROMPT CLAUDE CODE

Dans cloudflare/src/planif.js, metier.js, index.js et database.rules.json :

1. Budget de sous-requêtes vérifié AUSSI dans les boucles internes (envoi d'un défi à tous les athlètes, rappels 48 h, fins de coaches, ambassadeurs) : au-delà, écris la suite dans une sous-tâche par athlète et rends la main.
2. Un événement qui échoue passe en fin de file avec un compteur d'essais ; au 5e échec, il va dans evenements_ko/ (visible dans l'écran admin). Il ne bloque plus jamais les suivants.
3. Verrou pour minute() (bail court en base, 55 s) : deux exécutions ne traitent jamais la même file ensemble.
4. Règles : /evenements limité à un événement par type et par compte en attente, avec un intervalle minimal (at).
5. /sante?cles=1 protégé par un secret d'administration ; /reveil sans effet si la file a été traitée il y a moins de 30 s ; /arrivee et /amb-clic limités par IP (règles de limitation gratuites de Cloudflare, décrites dans le README).
6. Purge mensuelle de paypal_evenements de plus de 90 jours.
7. Mesure : combien de sous-requêtes et de temps CPU pour 10, 100, 1 000 abonnés push ; si le chiffrage des push dépasse 10 ms, envoie-les par petits lots d'une exécution à l'autre.
8. Tests de charge simulés dans cloudflare/test.

La page d'accueil annonce d'autres prix que l'app

CONSTAT

L'app (tableau OFFRES, décision du 24/09) : Essentielle 9,50 €/mois, Ultime 24,90 €/mois, annuel = 12 × mensuel, engagement 12 mois. La page d'accueil index.html affiche encore **9,95 € / 8,25 €** (99 € par an), Ultime 20,75 € (249 €/an) et « sans engagement », y compris dans la FAQ. Faire de la pub vers une page qui promet d'autres prix, c'est des remboursements, des litiges et des avis négatifs.

L'IDÉE

Aligner la page d'accueil sur OFFRES, idéalement en lisant les prix depuis une seule source pour que ça ne se reproduise plus. Mettre en avant ce qui est vrai et fort : 1 mois d'essai sans carte, 2 mois avec un parrain.

PROMPT CLAUDE CODE

Aligne les prix affichés partout sur le tableau OFFRES de app/rc-core.*.js :

1. Extrais les prix (Essentielle, Ultime, annuel, essai, essai parrainé, offre ULTIME_DEMI, formules coach Libre / Coach 19 € / Pro 39 €) dans un petit fichier partagé tarifs.json à la racine, lu par l'app et par index.html (et aide-apk.html, terms.html si des prix y figurent).
2. index.html : corrige la grille (9,50 € et 24,90 €, annuel 114 € et 298,80 €), retire « sans engagement » de la grille et de la FAQ, et explique l'engagement de 12 mois en une phrase claire.
3. Mets en avant : « 1 mois d'essai, toute l'app ouverte, sans carte » et « 2 mois si un ami t'invite ».
4. Ajoute un test dans app/tests.js (ou scripts/verif) qui échoue si un prix de index.html diffère de tarifs.json.
5. Vérifie terms.html : conditions d'engagement et de résiliation cohérentes avec OFFRES.

Les accrocs restants (notifications et partage)

CONSTAT

Toujours présents depuis la V3, maintenant côté serveur léger : « ta série est en danger » envoyé chaque jeudi, à vie, aux gens qui ont arrêté ; rappel de bilan à ceux qui n'ont ni coach ni bilan ; la carte promet « un rappel, rien d'autre » mais abonne aux 7 types ; les notifications locales d'Android ne comptent pas dans le plafond. Côté partage : sur iPhone, « Télécharger » ne copie pas le lien, le fichier s'appelle toujours repcore-seance.png, et le message de parrainage a une faute (« RepCore Avec mon code »).

L'IDÉE

Tout corriger d'un coup, avant que les notifications partent à des milliers de personnes.

PROMPT CLAUDE CODE

```
Corrige (cloudflare/src/metier.js planifies.*, app/sw.js, app/rc-core*.js) :
1. planifies.serie : recalcule la série à la date du jour (même logique que updateStreak/_streakPerime)
; rien si elle est cassée ou si la dernière séance date de plus de 14 jours.
2. planifies.bilan : seulement si l'athlète a un coach ET a déjà fait au moins un bilan.
3. Carte d'accord (etatInvitationNotif, invNotifOui) : 3 cases (Mes séances et ma série / Mon coach /
Mes invitations) pré-cochées selon le profil, écrites dans u.pushPrefs avant l'abonnement ; le texte dit
exactement ce qui sera envoyé.
4. Plafond commun : si le push serveur est actif, coupe les doublons locaux de sw.js (série, bilan,
wrapped) ; sinon les notifications locales respectent le même maximum d'une par jour.
5. _storySortirTelechargement : _storyCopierLien(src) avant la branche iOS (dans le même geste
utilisateur).
6. _ouvrirApercuStory : nom et type de fichier réels (visuelNomFichier, visuelFondFormat).
7. parrainageMessage : « Je m'entraîne avec RepCore. Avec mon code ... ». Relis tous les textes de
partage.
8. « Partager ce record » : un vrai bouton, ou un bouton « Partager mes records » au-dessus de la liste.
9. Tests : cloudflare/test/metier.test.mjs pour 1 et 2, app/tests.js pour 3 à 6.
```

A. Partage et acquisition

IDÉE 09 · IMPACT TRÈS FORT · EFFORT FAIBLE

« Thomas t'invite » : l'arrivée par un ami

CONSTAT

Le parrainage tourne, mais celui qui clique sur le lien d'un ami arrive sur /i, qui lui parle d'un code donné par « ton coach ». À l'inscription : « Invité par un ami ». Le prénom du parrain est pourtant stocké dans codesPublics/<code>.

L'IDÉE

« **Thomas t'invite sur RepCore** · 2 mois pour essayer », son emblème de rang, un seul bouton. Le prénom revient à l'inscription et au premier écran, et Thomas est prévenu à la première séance de son filleul.

PROMPT CLAUDE CODE

Accueil nominatif du filleul :

1. i/index.html : avec ref=<CODE>, lis /parrainage/codesPublics/<CODE> (prénom + rang seulement, vérifie les règles) et affiche « <Prénom> t'invite sur RepCore », l'emblème du rang, « 2 mois pour essayer », un bouton « Commencer ». Avec amb= : le nom de l'ambassadeur. Avec coach= : texte actuel. Sans rien : texte générique sans « code coach ».
2. Navigateur interne d'Instagram : garde l'échappement, avec le prénom.
3. parrainageChampInscription : « Invité par <Prénom> · 2 mois pour essayer ».
4. Premier écran : « <Prénom> sera prévenu quand tu feras ta première séance », et push au parrain à ce moment-là (type filleul, via cloudflare/src/metier.js).
5. iOS installé : si ref s'est perdu, « Quelqu'un t'a invité ? Entre son code » en haut de l'inscription.

IDÉE 10 · IMPACT FORT · EFFORT FAIBLE

Story et post 4:5 pour tous les visuels, et le Wrapped en carrousel

CONSTAT

Les visuels athlète (bilan, record, badge, Wrapped, muscles, cycle, série, rang) restent en 9:16 : postés dans le fil, ils sont recadrés et perdent titre et signature.

L'IDÉE

Un choix « Story / Post » à côté du choix de fond, avec une vraie mise en page 1080×1350. Le Wrapped en carrousel de 5 images 4:5 : le format le plus puissant du fil Instagram.

PROMPT CLAUDE CODE

Dans app/rc-core.*.js :

1. Réglage commun VISUEL_FORMATS {story:1080x1920, post:1080x1350} (généralise AA_FORMATS / VC_FORMATS) et sélecteur « Story / Post » à côté du sélecteur de fond sur tous les visuels.
2. Chaque fonction de dessin recalcule sa mise en page en 1350 px (pas de simple recadrage) ; pour un long bilan, « +N autres ».
3. Wrapped : « Carrousel pour mon fil » = les 5 slides en 4:5 partagées en une fois (navigator.share multi-fichiers, sinon une par une).
4. Nom de fichier repcore-<type>-post.jpg ; dernier format mémorisé (try/catch).
5. Tests : chaque visuel dans les deux formats avec des données extrêmes, rien ne déborde.

IDÉE 11 · IMPACT MOYEN · EFFORT FAIBLE

Une légende prête à coller, et #RepCore

CONSTAT

Aucun partage ne propose de texte : impossible de retrouver les publications des utilisateurs, ni de les reposter.

L'IDÉE

En Story, on copie le lien (sticker Lien). En Post, on copie une légende courte tirée des stats, avec #RepCore, le compte officiel et « lien dans ma bio ».

PROMPT CLAUDE CODE

Dans app/rc-core.*.js :

1. `legendePartage(type, donnees)` : 3 à 5 modèles par visuel, tutoiement, un $\times$ au plus, jamais deux fois le même de suite, toujours « #RepCore » + `RC_COMPTE_INSTAGRAM` (constante à remplir) + « Lien dans ma bio » si page publique active.
2. `_storySortirPartage / _storySortirTelechargement` : en Post, copie la légende et toast « Légende copiée · colle-la sous ta photo » ; en Story, garde la copie du lien. Passe aussi la légende dans `navigator.share(text)`.
3. « Voir la légende » : l'afficher et la modifier avant copie.
4. Tests : moins de 220 caractères, #RepCore présent.

IDÉE 12 · IMPACT TRÈS FORT · EFFORT MOYEN

L'export vidéo de la foudre (stories et Reels)

CONSTAT

Tout est image fixe alors que la foudre du record est spectaculaire. Sur Instagram, la vidéo porte beaucoup plus loin.

L'IDÉE

Un bouton « Vidéo » : 5 à 8 s, l'éclair frappe, la charge grésille, la carte se pose. Sans musique : l'utilisateur ajoute un son tendance dans Instagram.

PROMPT CLAUDE CODE

Dans app/rc-core.*.js :

1. `exporterVideoVisuel(scene, dureeMs)` : dessin image par image dans un canvas 1080x1920 (tracé de `rcFoudre` et `arcCompteur` pilotés par le temps de la vidéo), `canvas.captureStream(30)` + `MediaRecorder` ; `video/mp4` si supporté (Safari iOS 14.5+), sinon `webm vp9`. Débit ~4 Mbit/s, fichier < 8 Mo.
2. Scènes : record, passage de rang, `Wrapped` (5 slides, 1,5 s chacune), fin figée 1 s.
3. Bascule « Image / Vidéo » dans les écrans de partage, barre de progression, sortie par `navigator.share` ou téléchargement ; bouton caché sans `MediaRecorder`.
4. Test manuel documenté : iPhone (app installée), Android, publication en story et en Reel.

IDÉE 13 · IMPACT FORT · EFFORT FAIBLE

Le parrainage là où l'émotion est forte

CONSTAT

Le parrainage n'apparaît que dans le profil et en une ligne après un record. On invite quand on est fier : au passage de rang, à un palier de série, sur le Wrapped.

L'IDÉE

Une carte visuelle « Rejoins-moi sur RepCore » (rang, série, « 2 mois pour essayer », code en gros) et un bouton « Inviter un pote » sur ces écrans. Sur l'accueil : « 2 potes invités · 1 mois gagné ».

PROMPT CLAUDE CODE

```
Dans app/rc-core.*.js :  
1. _dessinerCarteInvitation 1080x1920 et 1080x1350, même épure que les autres visuels ; au partage, lien avec ref (story) ou légende avec le code (post).  
2. htmlParrainage : la carte en tête (« Partager ma carte d'invitation »), le texte et le lien en secondaire.  
3. Bouton secondaire « Inviter un pote » sur _celebrerRang, les paliers de série et la dernière slide du Wrapped.  
4. Accueil : une ligne sous le rang dès le premier filleul. Écran parrainage : état de chaque filleul par prénom (inscrit / 1re séance / abonné).
```

IDÉE 14 · IMPACT FORT · EFFORT MOYEN

Page publique riche, et aperçu de lien personnalisé

CONSTAT

La page /@pseudo montre l'emblème mais des badges en texte et des records sans valeurs, et personne n'est invité à l'activer. L'aperçu personnalisé (image avec le prénom et le rang) n'a pas été porté sur le serveur léger : tous les liens partagés en DM montrent l'image par défaut.

L'IDÉE

Une vraie vitrine (emblème, jauge, série, badges en images, 3 records au choix), proposée juste après le premier passage de rang, et un aperçu de lien généré par le worker.

PROMPT CLAUDE CODE

```
Page publique et aperçu (p/index.html, c/index.html, cloudflare/src, réglages htmlReglagesPagePublique) :  
1. Contenu : emblème en grand, jauge de volts, 12 dernières semaines de série, badges en images (secrets si découverts), 3 meilleurs records avec valeurs si l'athlète coche la case (décochée par défaut), nombre de séances. Jamais de poids, photos ou santé.  
2. Proposition d'activation à la fin de _celebrerRang (pseudo pré-rempli, un geste), rappel une fois au rang suivant.  
3. Aperçu de lien : route du worker /@<pseudo> et /coach/<slug> qui renvoie le HTML de la page avec des balises og:title/og:description/og:image personnalisées (images rang_N-og.jpg existantes + prénom + rang), mise en cache  
6 h ; firebase.json redirige ces chemins vers le worker (ou le domaine du worker est utilisé dans lienPerso).  
4. Vérifie l'aperçu dans WhatsApp et les DM Instagram.
```

B. Les nouveautés qui feront parler

IDÉE 15 · IMPACT TRÈS FORT · EFFORT MOYEN

La carte d'athlète notée sur 99

CONSTAT

Toujours aucune « carte de joueur ». C'est un des formats les plus partagés du sport, et il colle aux athlètes qui veulent juste leurs stats.

L'IDÉE

5 notes sur 99 calculées sur les vraies données (FOR, VOL, RÉG, PRO, END), une note globale, l'emblème de rang ; mise à jour chaque lundi (« 71 → 73 »), partage en story, vignette de la page publique. Le gabarit se fait sur ChatGPT (prompt de la V3, inchangé).

PROMPT CLAUDE CODE

Ajoute la carte d'athlète dans app/rc-core.*.js :

1. noteAthlete(u) -> {FOR, VOL, REG, PRO, END, globale} sur 1..99, sur 12 semaines ; FOR = e1RM des mouvements de base rapporté au poids de corps (barèmes homme/femme) ; REG et PRO pèsent plus que FOR ; courbe douce (dur de dépasser 90).
2. Gabarits cartes-bruts/ (standard, elite, legendaire) convertis en webp dans app/img/cartes/ ; cadre selon la note.
3. _dessinerCarteAthlete 1080x1512 (et story 1080x1920), signature REPCORE.
4. Recalcul le lundi (u.carte + historique) ; si la note monte : carte sur l'accueil + partage, rcFoudre au changement de cadre.
5. Onglet evolution : la carte et la courbe de la note ; page publique : vignette si activée. Tests des barèmes.

IDÉE 16 · IMPACT TRÈS FORT · EFFORT MOYEN

Le duel entre potes (et un défi du mois pour les autonomes)

CONSTAT

Les défis n'existent que dans le Canal d'un coach : un athlète autonome n'a ni défi ni badge CHAMPION. Pourtant le défi entre amis est le mécanisme viral le plus naturel : pour relever le défi, le pote installe l'app.

L'IDÉE

« Défie un pote » sur 7 ou 14 jours, en régularité ou en progression %, jamais en charges brutes. Le lien porte le code parrain. Carte « DUEL » à partager au lancement et au résultat.

PROMPT CLAUDE CODE

Duels et défi ouvert, avec le serveur léger :

1. /duels/<id> {createur, invite, mesure, duree, debut, statut, scores} : lecture par les deux participants seulement, scores écrits par le worker (règles database.rules.json).
2. cloudflare/src/metier.js : calcul des scores à chaque séance terminée (réutilise la logique des défis du Canal), dans le budget de sous-requêtes (voir idée 06).
3. Bouton « Défie un pote » (accueil, écran parrainage) -> lienPerso()?duel=<id> (garde ref) ; arrivée nominative (idée 09) « <Prénom> te défie : 14 jours de régularité » ; démarrage à la 1re séance de l'invité.
4. Cartes « DUEL » (lancement, résultat), badge CHAMPION au gagnant.
5. Défi RepCore du mois pour tous (créé par Kevin depuis l'écran admin), affiché aux autonomes.
6. Push : début, J-2, résultat. Tests dans cloudflare/test.

Les éditions limitées

CONSTAT

Aucun temps fort saisonnier. La rareté datée (« plus jamais disponible après le 31 janvier ») est ce qui fait le plus poster.

L'IDÉE

4 à 6 événements par an (Janvier Électrique, Printemps Titan, Été Sans Pause, Rentrée Foudre, Décembre Inarrêtable), chacun avec un défi ouvert, un badge daté introuvable ensuite, un fond de story exclusif et un compteur collectif. Badges : prompt ChatGPT de la V3.

PROMPT CLAUDE CODE

Événements saisonniers (le nom /evenements est déjà pris par la file du worker : utilise /saisons) :

1. /saisons/<id> {nom, debut, fin, mesure, objectifPerso, objectifCollectif, badgeCle, couleurAccent, texteAccueil}, créés par Kevin depuis l'écran admin ; progression calculée par le worker.
2. Bannière d'accueil avec compte à rebours, jauge perso, compteur collectif (/stats/saisons/<id>, mis à jour chaque heure par le worker, dans son budget).
3. Famille de badges « Éditions » avec l'année ; « Plus jamais disponible » après la fin.
4. Fond de story « Édition » pour tous les visuels pendant l'événement.
5. Push : lancement, mi-parcours pour les retardataires, J-2, fin. Carte « J'ai bouclé <événement> ».

C. Addiction et rétention

IDÉE 18 · IMPACT FORT · EFFORT FAIBLE

Une seule fête par séance

CONSTAT

Une séance peut encore enchaîner jusqu'à cinq écrans pleins (fin, série, badges, récap, rang). Au troisième, on ferme sans lire.

L'IDÉE

Le moment le plus rare en plein écran (rang > palier IV > secret > série > badge), tout le reste dans un carrousel « Tes trophées du jour » sous le bilan, chacun partageable.

PROMPT CLAUDE CODE

```
Dans app/rc-core.*.js, file _bdgFile :  
1. Classe les événements de la séance par rareté ; un seul écran plein avec rcFoudre ; BDG_ECRAN_MAX = 1.  
2. Le reste dans un carrousel « Tes trophées du jour » sous l'écran de fin (vignettes, Partager, fiche au tap).  
3. Rattrapages : un seul récap, sans foudre. Supprime l'affichage « Plafond du jour atteint ».  
4. Réglage « Célébrations : complètes / discrètes ». Tests de la priorisation.
```

IDÉE 19 · IMPACT TRÈS FORT · EFFORT MOYEN

Les 7 premiers jours : « Mise sous tension »

CONSTAT

Le premier badge demande 10 séances : un nouveau ne débloquent presque rien la première semaine, et c'est là que tout se joue, surtout avec 1 mois d'essai gratuit à convertir.

L'IDÉE

7 étapes sur l'accueil (1re séance, 1er record, profil, notifications, 2e séance, 1re semaine, inviter un pote), des volts à chaque étape, et le badge unique SOUS TENSION en une semaine environ.

PROMPT CLAUDE CODE

```
Dans app/rc-core.*.js :  
1. PARCOURS_DEMARRAGE : 7 étapes, test et volts (hors plafond du jour, une seule fois) ; « inviter un pote » seulement si PARRAINAGE_ACTIF.  
2. Carte « Mise sous tension · 3/7 » en haut de l'accueil pendant 14 jours ou jusqu'à la fin.  
3. Badge unique sous_tension (badges-bruts/sous_tension.png, prompt ChatGPT de la V3), écran plein, carte partageable. Attention : « SOUS TENSION » existe déjà comme mesure de temps sous tension dans la séance ; nomme la clé parcours_sous_tension et vérifie qu'aucun libellé ne se mélange.  
4. Rattache le parcours à l'essai : au jour 21, si le parcours n'est pas fini, push « encore 2 étapes » (type serie).  
5. Comptes existants : parcours considéré comme terminé. Tests.
```

IDÉE 20 · IMPACT TRÈS FORT · EFFORT FAIBLE

« Record à portée »

CONSTAT

L'app connaît la tendance de chaque exercice mais ne dit jamais ce qui est atteignable à la prochaine séance.

L'IDÉE

« > Record à portée : squat 105 kg × 5 (record : 102,5 × 5) » sur l'accueil et en tête de séance ; « OBJECTIF ATTEINT » sur la carte record s'il tombe ; utilisé dans le rappel du jour de séance.

PROMPT CLAUDE CODE

Dans app/rc-core.*.js :

1. recordAPortee(u, seancePrevue) : tendance eIRM des 6 dernières séances par exercice, plafonnée à +2,5 % du record, arrondie au pas de charge ; rien si tendance en baisse, décharge, ou phase de cycle à charge réduite.
2. Affichage accueil + en tête de s-workout ; petit éclair sur la série concernée.
3. Carte record : sur-titre « OBJECTIF ATTEINT ». Rappel de séance (push ou local) avec ce texte.
4. Tests : hausse, baisse, pas de données, pas de 1,25 / 2,5 kg.

IDÉE 21 · IMPACT FORT · EFFORT MOYEN

Check-in du matin et relance des inactifs

CONSTAT

Un jour de repos n'offre presque rien à faire. Et rien ne relance un athlète qui décroche : maintenant que le push part vraiment, c'est le levier le moins cher.

L'IDÉE

Un check-in en 3 taps (sommeil, énergie, courbatures) qui donne une « batterie du jour ». Et une relance courte et personnelle à J+7, J+14 et J+30, puis silence.

PROMPT CLAUDE CODE

Dans app/rc-core.*.js et cloudflare/src :

1. Check-in du matin sur l'accueil jusqu'à 14 h, enregistré dans /users/<id>/checkin/<date> ; batterieDuJour -> 0..100 % + phrase ; +10 V (dans le plafond) ; visible par le coach.
2. Nouveau type push 'retour' (PUSH_TYPES, réglages) ; tâche quotidienne 11:00 dans cloudflare/src/planif.js : dernière séance à J+7, J+14 ou J+30, hors suspension, 3 messages max par période ; textes avec ses vraies données (rang, tonnage + equivalentTonnage, reprise -10 %).
3. J+30 : écran « Reprise en douceur » (baisse de 10 % proposée, jamais automatique).
4. Retour après 10 jours : écran « RETOUR AU COMBAT » + progression vers PHÉNIX NOIR.
5. Pas de doublon avec la relance série. Tests dans cloudflare/test.

IDÉE 22 · IMPACT MOYEN · EFFORT FAIBLE

Sous-niveaux de rang et volts vérifiés par le serveur

CONSTAT

Après ÉLITE, 6 mois et plus entre deux rangs sans rien entre. Une séance d'une minute vaut 100 V, et le coach lit un total écrit par le client. Avec le serveur léger, on peut enfin le vérifier.

L'IDÉE

Trois sous-niveaux par rang (MONSTRE I, II, III) avec une petite célébration, des volts selon le contenu réel, et un total recalculé par le worker.

PROMPT CLAUDE CODE

Dans app/rc-core.*.js et cloudflare/src/metier.js :

1. Sous-niveaux I/II/III à partir de VOLTAGE ; toast + petite foudre ; chevrons dessinés sous l'emblème.
2. 100 V seulement si ≥ 15 min et ≥ 6 séries validées, sinon séries × 10 (max 100).
3. Le worker recalcule le total à chaque séance terminée (événement existant) et écrit /xp_serveur/<compte> (écriture worker seule) ; coach, page publique et défis lisent cette valeur.
4. Dates des badges secrets contrôlées avec l'heure du serveur. Tests.

D. Influence et pilotage

IDÉE 23 · IMPACT FORT · EFFORT FAIBLE

Le kit des coachs et ambassadeurs, et l'offre de lancement

CONSTAT

Les codes ambassadeurs et les commissions tournent, mais personne ne donne aux coachs et influenceurs de quoi publier chaque semaine. L'offre ULTIME_DEMI (1er mois à moitié prix) existe dans PayPal mais n'est reliée à rien de visible.

L'IDÉE

Un onglet « Mon kit » avec 3 contenus prêts chaque lundi (recap team, victoire, défi), les éléments de marque, et pour les ambassadeurs leurs stats et une carte à leur nom. ULTIME_DEMI devient l'avantage des codes ambassadeurs pendant le lancement.

PROMPT CLAUDE CODE

Dans l'espace coach et la page ambassadeur :

1. Chaque lundi, 3 contenus 1080x1350 + légendes (réutilise _dessinerRecapTeam, _dessinerVictoireCoach avec consentement, défi en cours), boutons « Tout télécharger » et « Copier la légende ».
2. Éléments de marque : logo clair/sombre, 3 fonds, emblèmes de rang, 3 règles d'usage.
3. Page ambassadeur (lien secret) : stats de la semaine, lien, carte d'invitation à son nom, 5 idées de publications.
4. Offre de lancement : un code ambassadeur peut porter l'avantage 'ultime_demi' (1er mois Ultime à moitié prix, plan PayPal ULTIME_DEMI) au lieu du mois d'essai supplémentaire ; affiché à l'arrivée (idée 09) et à l'inscription.
5. Rappel légal dans la page ambassadeur : mention « Partenariat commercial » obligatoire (loi de 2023).

IDÉE 24 · IMPACT FORT · EFFORT FAIBLE

Mesurer la rétention

CONSTAT

L'écran Viralité montre l'entonnoir par source et le coefficient viral, mais pas si les gens restent. Or c'est la rétention qui dit si l'app est addictive, et si l'essai d'un mois se transforme.

L'IDÉE

Cohortes J1/J7/J30, utilisateurs actifs par jour et par semaine, taux de conversion essai → payant, et l'effet de chaque levier (parcours, duel, joker, partage, push).

PROMPT CLAUDE CODE

Complète ouvrirViralite (CREATOR_EMAIL) avec une tâche quotidienne du worker (dans son budget, par lots) :

1. /stats/retention : par semaine d'inscription, % actifs à J1, J7, J30 ; DAU, WAU, MAU, DAU/MAU.
2. Entonnoir : inscription -> 1re séance -> parcours fini -> fin d'essai -> payant (par source src/ref/amb).
3. Effet des leviers : rétention J30 avec et sans chaque levier, taille des groupes, alerte sous 30 personnes.
4. Tableaux et courbes en SVG léger, aucune donnée personnelle. Mets à jour privacy.html si besoin.

Le calendrier de contenu du compte RepCore

CONSTAT

Le compte officiel n'a pas encore de rythme, alors que l'app produit maintenant des visuels chaque jour.

L'IDÉE

Un rythme simple et un premier mois préparé d'avance par ChatGPT, à partir des vraies fonctions de l'app.

PROMPT CHATGPT — CALENDRIER ÉDITORIAL

Tu es responsable des réseaux sociaux de RepCore, une application française de musculation (PWA) : noir et rouge, univers tonnerre et électricité, tutoiement, public 18-40 ans qui s'entraîne en salle. À mettre en avant : bilan de séance à partager en story, foudre sur les records, 50 badges dont 10 secrets (indices : « Le fer est plus froid avant le soleil », « Trois éclairs, un seul orage »), 10 rangs (Étincelle, Impulsion, Voltage, Machine, Élite, Surtension, Monstre, Foudre, Titan, Légende), Wrapped mensuel, défis du coach, parrainage (2 mois pour essayer, 1 mois offert au parrain), essai d'un mois sans carte, prix 9,50 € et 24,90 € par mois. Crée un calendrier Instagram de 4 semaines : lundi carte de la semaine, mardi badge secret, mercredi Reel d'un record, jeudi coach à l'honneur, vendredi repost d'un athlète, week-end défi. Pour chaque publication : format, accroche des 2 premières secondes, légende (max 150 mots), 5 hashtags dont #RepCore, idée de visuel précise, appel à l'action. Ajoute le lancement d'une chaîne de stories « Ajoute le tien » et 3 idées de Reels adaptables au son du moment. Présente le tout en tableau.

Règles d'or

- Avant toute campagne : prix identiques partout, et les deux bugs serveur (01, 02) corrigés.
- Un visuel = un chiffre héros, « NOM · REPCORE », jamais de QR. Story : lien copié. Post : légende copiée.
- Une notification doit être vraie, personnelle et rare : au plus une par jour, toutes sources confondues.
- Tout ce qui donne de l'argent ou un accès est écrit par le serveur, jamais par le client.
- On compare la régularité et la progression, jamais les charges brutes.
- Chaque nouvelle fonction se juge sur la rétention à 30 jours.